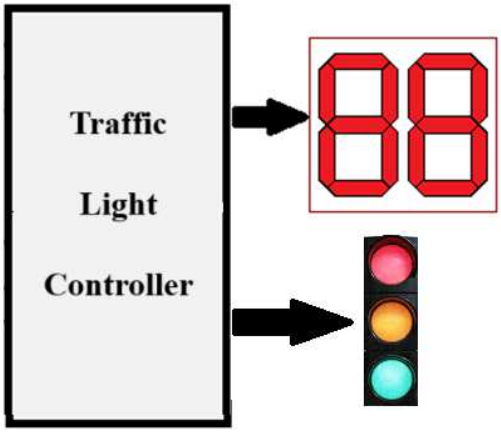
An Najah National University
Faculty of Engineering
Computer Engineering Department
Digital design 2 LAB -10636391-
Experiment 4: Algorithmic State Machine (*Traffic Light*)
Eng. Alaeddin Minwer , Dr. Anas Toma and Dr. Ashraf Armoush

Lab Overview:

In this lab, you will implement an algorithmic state machine (ASM) on the ZedBoard to implement a traffic light controller with a countdown timer.

Specification:

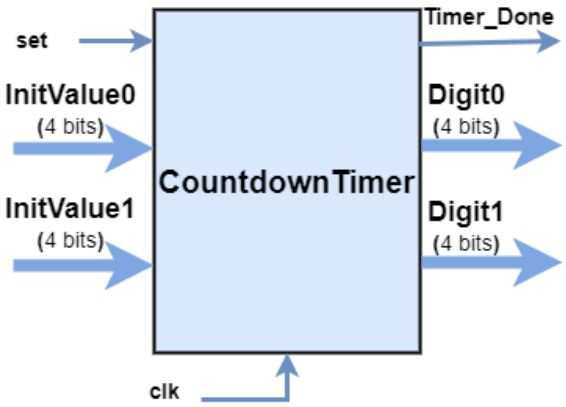
The system described in this experiment is designed to illustrate the concept and fundamental operations of a traffic light controller. The system is using two 7-segment components to display the countdown timer for each state and three LED's to perform the traffic lights operation.



Lab Procedure:

Part 1: Two-Digit BCD Countdown Timer

A countdown timer counts the time in backwards to show the amount of time left and stops counting when the counts reaches zero. It consists of two digits and count the time in seconds.



```
entity CountdownTimer is
  Port (
      clk          : in  std_logic;
      set          : in  std_logic;  -- async set to load the initial value
      InitValue1   : in  std_logic_vector(3 downto 0);
      InitValue0   : in  std_logic_vector(3 downto 0);
      Digit1       : out std_logic_vector(3 downto 0); -- tens digit
      Digit0       : out std_logic_vector(3 downto 0); -- ones digit
      Timer_Done   : out std_logic
  );
end CountdownTimer;
```

1. Complete the architecture to implement the countdown timer.
2. Write a testbench to simulate the timer and make sure it behaves correctly.

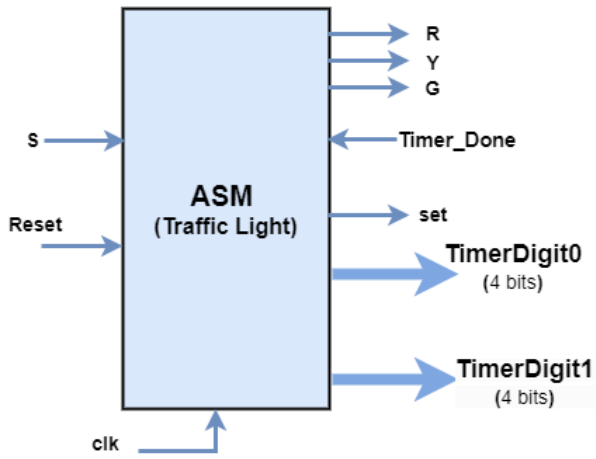
Part 2: Clock Divider:

The countdown timer in part 1 is counting the time in seconds and the system is operating on 100MHz frequency. Therefore, this entity is used to divide the input clock to generate an output clock with 1Hz frequency.

```
entity Divider is
    Port(   CLK_IN      : in  STD_LOGIC;
           CLK_OUT      : out STD_LOGIC);
end Divider;
```

Part 3: ASM Simulation:

In this part you will implement the traffic light controller using an algorithmic state machine. Your system should implement the ASM chart given in the next page.

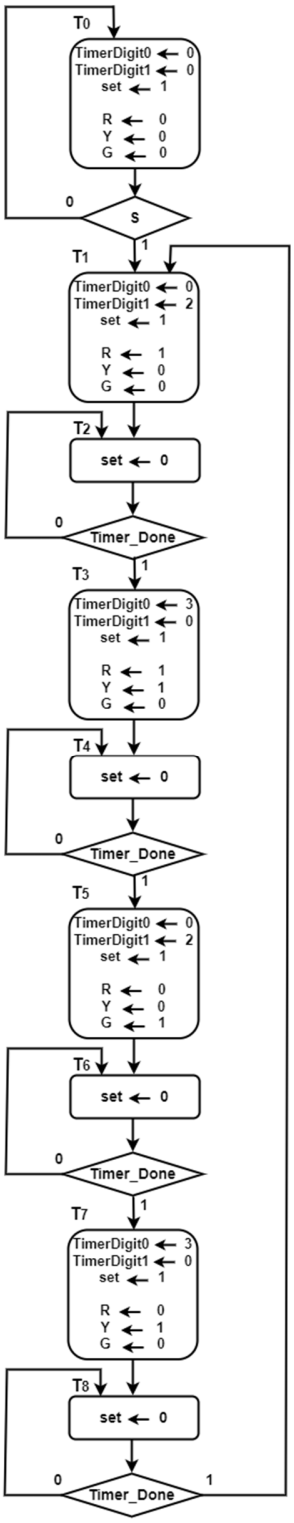


Experiment 4: Algorithmic State Machine (*Traffic Light*)

Eng. Alaeddin Minwer , Dr. Anas Toma and Dr. Ashraf Armoush

The traffic light controller provides a repeated sequence that consists of 4 stages:

Sequence	R	Y	G	Duration
a) Red	1	0	0	20 seconds
b) Red + Yellow	1	1	0	3 seconds
c) Green	0	0	1	20 seconds
d) Yellow	0	1	0	3 seconds



1. Implement the given ASM chart in VHDL using 3 processes as shown below:
2. Simulate your code and make sure it behaves correctly

Define the ASM entity :

```
entity ASM is
      port (      clk, reset, s       : in     STD_LOGIC;
                  Timer_Done          : in     STD_LOGIC;
                  set, R, Y, G        : out     STD_LOGIC;
                  TimerDigit0         : out STD_LOGIC_VECTOR (3 downto 0);
                  TimerDigit1         : out STD_LOGIC_VECTOR (3 downto 0));
end ASM;
```

- Define the different states as enumeration data type.

```
type state_type is (T0, T1, ...... );
signal current_state, next_state :state_type ;
```

- You can divide your code into 3 processes
  - The **first process** will be used to perform the reset and changes on the rising edge of a clock signal (*clocked synchronous state machine*).

```
state_register : process( clk, reset)
begin=
      if(reset = '1') then
            current_state <= T0;
      elsif (clk'event and clk = '1') then
            current_State <= next_state;
      end if;
end process
```

  - The **second process** steps through each state based on state diagram (*ASM-Chart*).

```
next_state_process : process( s, current_State, Timer_Done)
begin
      case current_state is
      when T0 =>
            ...
      when T1 =>
            ...

end process
```

  - The **third process** controls the data-path and the output of the ASM . (*Note that the output should be synchronized with the clock*).

```
output_process : process( clk)
begin
      if(clk'event and clk = '1') then
            case current_state is
                  when T0 =>
            ...
                  when T1 =>
            ...
end process ;
```

Part 4: Push Button Debouncer

In this part you have to implement a debouncing module that would debounce the push buttons before they are connected to the ASM machine.

```
entity PBDebouncer is
    Port ( CLK           :  in STD_LOGIC;
           PB            :  in STD_LOGIC;--the button state to be debounced
           PB_debounced:  out STD_LOGIC);--the button state after debouncing
end PBDebouncer;
```

- The debouncing module is just a delay that ensures the correct state of the button. Try a delay for 10 ms.

Part 5: Time_MUX:

We cannot simultaneously display two digits on both 7-segment LED displays. Instead, by repeatedly and continuously display a digit on each display faster than the human eye can respond, both displays will appear to be illuminated at the same time. In this part, you will implement a multiplexer to select between the two digits (Tens and Ones) of the countdown timer. Each 7-segment LED display should be illuminated for **10 ms**. The output DigSelect signal is used to indicate the current active digit (0 and 1 for digit0 and digit1 respectively).

```
entity Time_MUX is
    Port (  clk         : in STD_LOGIC;
            Digit0      : in STD_LOGIC_VECTOR (3 downto 0);
            Digit1      : in STD_LOGIC_VECTOR (3 downto 0);
            BCD_Value   : out STD_LOGIC_VECTOR (3 downto 0);
            DigSelect   : out STD_LOGIC);

end Time_MUX;
```

Part 6: BCD_To_SevenSeg:

The BCD to 7 Segment Decoder converts a 4-bit BCD number to a 7-bit code which can be displayed on a 7-segment display. The seven segments in this module are common cathode. Therefore, your output signals should be active high.

(Hint: you can use a ***CASE*** statement to implement this decoder)

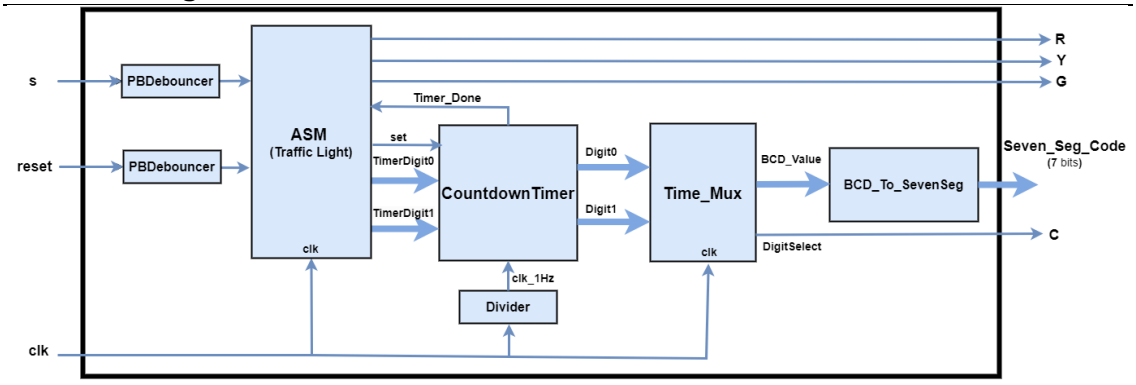
```
entity BCD_To_SevenSeg is
    Port ( BCD: in STD_LOGIC_VECTOR(3 downto 0);
           AA  : out  STD_LOGIC;
           AB  : out  STD_LOGIC;
           AC  : out  STD_LOGIC;
           AD  : out  STD_LOGIC;
           AE  : out  STD_LOGIC;
           AF  : out  STD_LOGIC;
           AG  : out  STD_LOGIC);
end BCD_To_SevenSeg;
```

Part 7: Top-Level Design

In this part you have to add the main top-level design which consists of the following components:

a. **PBDebouncer**: to **debounce** the signals from the push buttons.
b. **ASM**: the algorithmic state machine for the traffic light controller.
c. **Divider**: to generate a 1 Hz clock source from the input 100 MHz clock.
d. **CountdownTimer**: a countdown timer to be used by traffic light controller.
e. **Time_MUX**: to select between the two digits (Tens and Ones) of the countdown timer (TimerDigit1 and TimerDigit0).
f. **BCD_To_SevenSeg**: a BCD to seven-segment decoder.

```
entity Top_Level is
     port( clk              : in  STD_LOGIC;
           reset            : in  STD_LOGIC;
           s                : in  STD_LOGIC;
           R                : out STD_LOGIC;
           Y                : out STD_LOGIC;
           G                : out STD_LOGIC;
           Seven_Seg_Code   : out STD_LOGIC_VECTOR (6 downto 0);
           C                : out STD_LOGIC);
end Top_Level;
```



Synthesis, Implementation, and Bitstream Generation:

- Add a constraint file to assign the input ports as follows:
  - **clk**: Y9
  - **s**: Push Button
  - **reset**: Push Button
  - **R:** LD0.
  - **Y:** LD1.
  - **R:** LD2.

	Signal Name	Package Pin	I/O Std
Up Button	s	T18	LVCMOS18
Down Button	reset	R16	LVCMOS18

	Package Pin	I/O Std
clk	Y9	LVCMOS33

Outputs	Package Pin	I/O Std
LD0	T22	LVCMOS33
LD1	T21	LVCMOS33
LD2	U22	LVCMOS33

- Connect the seven-segment display module JA and JB connector, where you can use the following pin assignment:

Signal Name	Package Pin	I/O Std
JA1	Y11	LVCMOS33
JA2	AA11	LVCMOS33
JA3	Y10	LVCMOS33
JA4	AA9	LVCMOS33
JB1	W12	LVCMOS33
JB2	W11	LVCMOS33
JB3	V10	LVCMOS33
JB4	W8	LVCMOS33

1. Configure the FPGA on the ZedBoard board with the bit file.
2. Test your design with different test cases.